



피지컬 AI · AI Agent 설계 및 활용

CodeMaker

프레젠테이션

백준을 대체하는 Online Judge Platform

신우진, 심수민, 진우혁



목차

01 프로젝트 소개

02 프로젝트 내용

03 핵심 기술 및 알고리즘 소개

04 프로젝트 구현 현황

05 프로젝트 결과

06 Demo

개발 배경

백준의 서비스 종료

BOJ 채점 서비스 준비 중

2010년 3월 19일 - 2026년 4월 28일

그동안 이용해 주셔서 정말 감사합니다.

채점 서비스와 함께 곧 다시 돌아오겠습니다.

기존 학습 방식의 한계

- 개개인의 수준에 맞는 문제 선택이 어려움
- 문제 풀이 도중 막혔을 때 적절한 단계별 힌트 부족
- 기존의 시스템들은 대부분 문제은행 중심이라 개인 약점 반영 불가
- 오답, 시간초과, 런타임 에러 원인 분석 부담

프로젝트 목표

문제를 추천하는 수준을 넘어, 문제를 생성하고 검증하며, 피드백까지 제공하는 Agent 기반 통합 학습 플랫폼

주요 기능 소개

문제 자동 생성

알고리즘 및 난이도 기반의 맞춤형 문제 생성

다양한 Agent

생성 검증 분석 Agent가 역할

힌트 및 피드백 제공

정답을 바로 제공하는 것이 아니라,
학습자의 사고를 유도하는 힌트 및 피드백

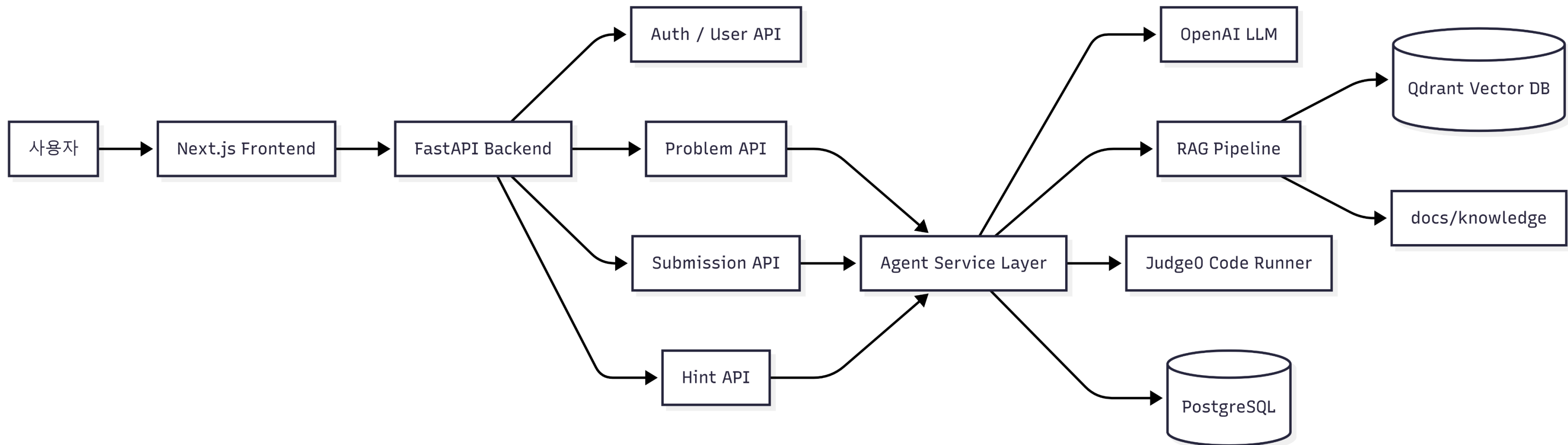
RAG 연동

알고리즘 개념 문서를 검색하여 LLM의 답변 보강

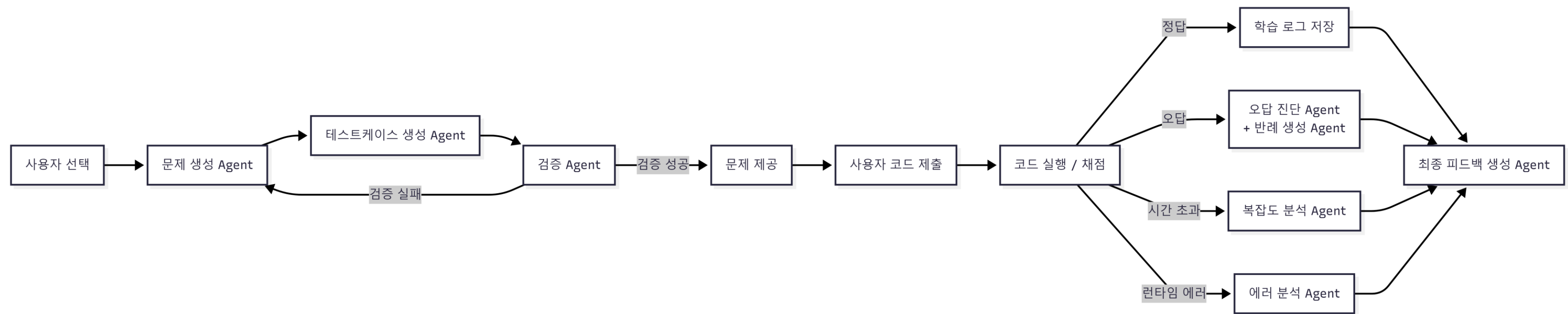
기존 시스템에 대한 차별점

기존 시스템	기존 시스템의 특징	한계	CodeMaker의 차별점
ChatGPT Study Mode	단계별 설명과 질문 중심 학습 지원	코딩테스트 전용 문제 생성 채점 구조 부족	코딩테스트 문제에 대해 생성, 제출, 실행, 오답 분석까지 연결
프로그래머스 AI 튜터 리포트	자료구조 알고리즘 영역 분석 후 문제 추천	기존 문제 은행 기반 추천 문제 수가 적음	사용자 조건에 맞는 신규 문제 생성
HackerRank 및 CodeSignal	코딩 면접 평가 상황 대비	학습보다는 평가 상황 대비	자기주도 학습 루틴 제공
Dometrain 및 AlgoLocal	AI 기반 문제 생성 및 코드 실행 지원	Agent 검증 workflow와 학습 로그 기반의 개인화 제한	Generator, Validator, Testcase Generator, Feedback Agent 분리

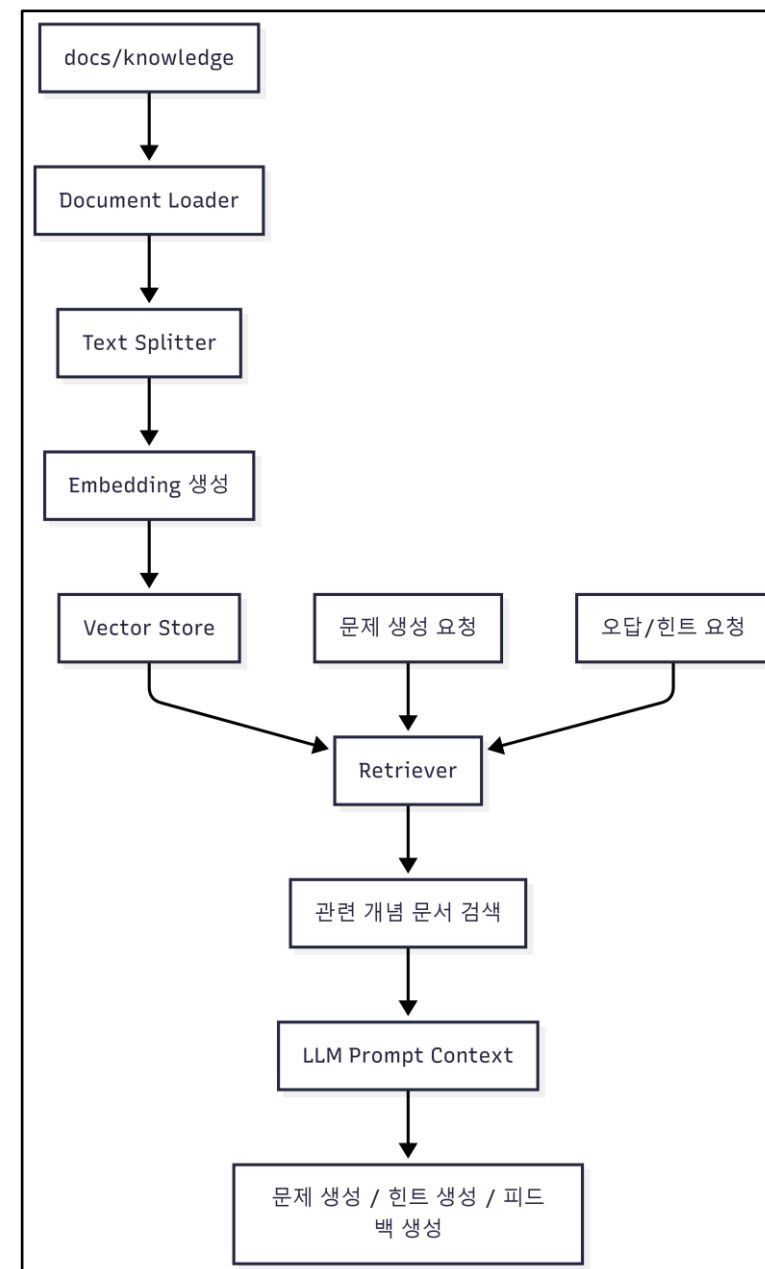
시스템 전체 구조



Agent workflow



RAG 기반 학습 지원



RAG 활용 지점

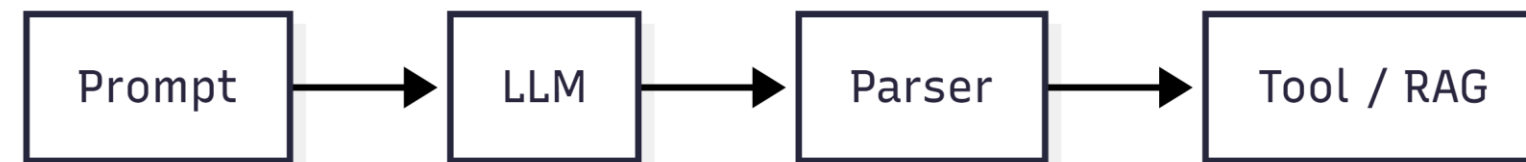
- 문제 생성: 알고리즘 개념, 난이도, 테스트케이스 전략 검색
- 힌트 생성: 문제별 저장 힌트 관련 개념 검색
- 오답 분석: 시간복잡도, 경계값, 입력 처리 문서 검색
- 피드백 생성: 검색된 개념을 바탕으로 학습 방향 제시

MVP에서의 활용

- Docs/knowledge 하위에 알고리즘 문서, 풀이 패턴 문서, 문제 생성 전략 문서를 구성하고 이를 임베딩

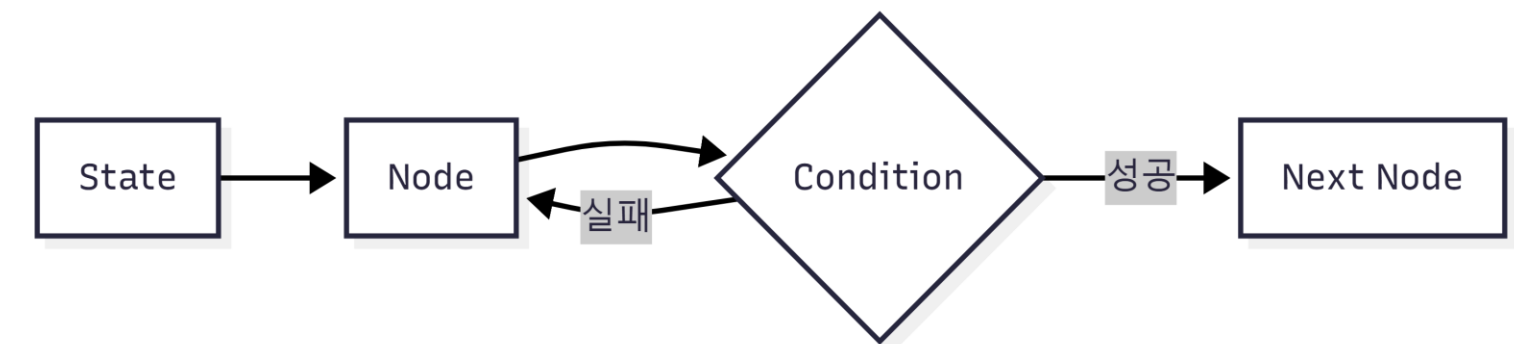
LangChain & LangGraph

LLM 연결과 Agent 실행 흐름 제어



LangChain

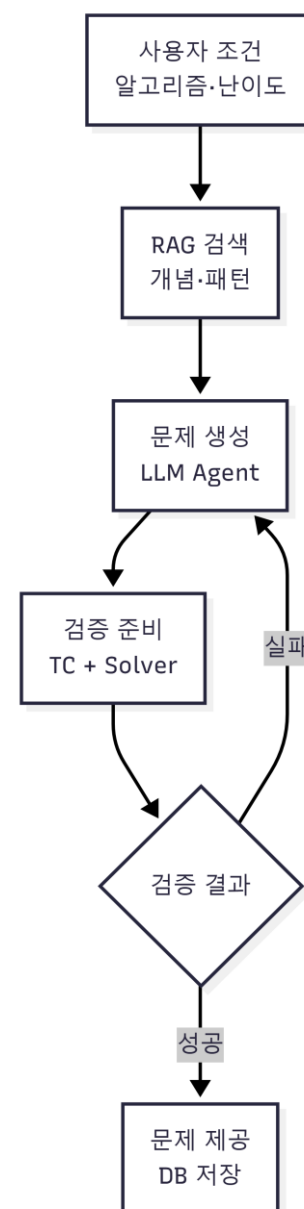
- LLM 호출
- PromptTemplate 관리
- Structured Output Parser
- RAG / Tool 연동



LangGraph

- StateGraph 기반 workflow 제어
- Node 단위 Agent 실행
- Conditional Edge 기반 분기
- 검증 실패 시 retry loop 구성

문제 생성 알고리즘



문제 생성 흐름

- 사용자가 선택한 알고리즘, 난이도, 학습 목표를 기반으로 생성 요청 구성
- RAG를 통해서 관련 알고리즘 개념과 풀이 패턴 문서를 검색
- LLM Agent가 문제 지문, 입출력 형식, 제한 조건, 예제 생성
- Seed와 variant를 이용해서 동일한 유형 안에서도 다양한 문제 형태 선택
- Testcase generator와 reference solver를 연결하여 정답 기준을 만듦
- Validator Agent가 테스트케이스와 정답 출력을 비교하여 문제 정합성을 검증
- 검증을 통과한 문제만 DB에 저장하고 사용자에게 제공

사용 기술 및 도구

영역	기술
FrontEnd	Next.js, TypeScript, Tailwind CSS
BackEnd	FastAPI, SQLAlchemy, Alembic
Agent	LangChain, LangGraph
LLM	OpenAI API
RAG	Embedding, Vector Store, Qdrant
Code 실행	Judge0
Database	PostgreSQL
Infra	Docker Compose

MVP에서의 사용

- Docker Compose 기반으로 API, Web, PostgreSQL, Qdrant, Judge0 실행
- 사용자 요청은 FastAPI 서버를 거침
- Agent는 LLM, RAG, Judge0, DB를 조합하여 문제 및 피드백 생성

초기 설계 vs MVP

항목	초기 설계	MVP 구현
문제 생성	다양한 알고리즘 전체 지원	Binary search, BFS, DFS 등
문제 다양화	사용자 약점 기반 동적 생성	see,d, force_new, avoid_problem_ids
문제 검증	Agent 기반 자동 품질 평가	Reference Solver + Testcase Generator + Judge0
피드백	장기 학습 이력 기반 개인화	제출 결과 기반 힌트/피드백
RAG	Qdrant 기반 영속 검색	Docs/knowledge 색인 + 인메모리 fallback

MVP 구현 결과 및 향후 과제

구현 완료

- 로그인 / 회원가입
- 문제 생성 화면
- 문제 목록 및 풀이 화면
- 문제 생성
- Seed 기반 문제 다양화
- 테스트케이스 생성
- Judge0 코드 실행
- RAG 문서 포함 및 색인
- Docker 기반 API / Web 실행

미해결 문제점

- 문제 생성 알고리즘 제한
- Qdrant 집합 자동 생성 개선 필요
- Qdrant 영속 색인 개선 필요
- 사용자 학습 기반 추천은 초기 단계
- 생성 문제 품질 평가 지표 부족
- 오답 분석 정확도 평가 필요
- 보안, 로그, 모니터링 보완 필요

기대효과

- 개인에 맞는 문제 제공
- LLM 생성 문제의 신뢰성 확보
- 오답 피드백을 통해 디버깅 향상
- 단계별 힌트를 통해 AI 의존 방지
- Workflow 기반 알고리즘 확장



감사합니다

+010-3672-0342
wlsdngur12@gmail.com
jinnuu.site

작성자 : 진우혁

핵심정리 페이지

핵심 요약과 실행 방향

핵심 요약은 프레젠테이션 전체 내용을 짧은 문장과 키워드 중심으로 정리하여 청중이 중요한 내용을 빠르게 되짚을 수 있도록 돕는 부분입니다. 이를 바탕으로 실행 포인트에서는 정리된 내용을 기반으로 다음에 취해야 할 행동이나 결정할 사항을 제시하여 단순한 요약에 그치지 않고 현실적인 실천 방향까지 이끌어냅니다.

문제 인식과 해결책

문제 인식은 발표 주제와 관련된 핵심 문제, 고민, 이슈를 간단하게 요약하여 청중에게 공감대를 형성하고 문제의 본질을 되짚어보는 역할을 합니다. 이어서 해결 방향에서는 제기된 문제에 대한 핵심 해결 방안이나 제안된 전략을 명확하게 정리해 변화의 가능성을 제시하며 발표의 흐름을 자연스럽게 마무리합니다.